Today

Welcome to your next lesson: Turning a response into data with .json()

I'm getting your lesson ready based on what you've learned, what's ahead, and the best way to help you grow.

Keep going, you're doing great.

Ron, let's connect this to what you just did with fetch and .then() / .catch() 🎯

So far, your code gets an HTTP response, not the actual data yet.

Think of it like ordering from Amazon:

The box = HTTP response
The thing inside the box = response body / content (the data you really want)

With fetch('https://jsonplaceholder.typicode.com/posts/1'), the first .then() gets the box (a Response object), not the unpacked data.

Quick check for you:

When fetch(...) resolves, what does the first .then() receive?

A: The final data already unpacked
B: A Response object (the "box") that still needs to be opened

You can just answer with A or B.

b

Exactly right — B 🎯

The first .then() gets a Response object (the box), not the actual data yet.

Now, most APIs (including jsonplaceholder.typicode.com) send the inside of the box as JSON.

Very simple idea:

JSON is just text that looks like a JavaScript object, and can be turned into a real JS object.

Example shape (don't overthink it yet):

Json
{

```
  "id": 1,
  "title": "Hello",
  "body": "World"
}
```

This is JSON text, which we'll turn into a JS object in a moment.

Does that basic idea of JSON-as-text-that-looks-like-an-object make sense so far? ☑ / ?

Yes, that makes sense

Great 👍

Next step: we need to open the box (the Response) and turn that JSON text into a real JS object.

In JavaScript, the Response object has a method called .json().

Step by step:

First .then() gets response (the box).
Inside that .then(), we call response.json().
response.json() will eventually give us the real data (as a JS object).

One important thing:

response.json() does not give the data back immediately. It returns another Promise, so we need a second .then() to get the actual data.

Before we touch code, I want to check your mental picture:

Would you say the second .then() in the chain receives:

A: The Response object
B: The parsed data (like a normal JS object)

Answer with A or B.

B

Nice, exactly — B 🎯

The second .then() gets the parsed data (a normal JS object), not the Response.

Let's wire this up in real code now, step by step.

I'd like you to do this in the code editor, not in chat:

Open the editor (</> button).
In JavaScript, type this skeleton:
Javascript

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    console.log('Got response:', response);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

Hit Run to just see what response looks like.
Then hit Submit code so I can see it.
Do that first — don't add .json() yet.

Once you've submitted, I'll help you add the .json() and the second .then().

Javascript

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    console.log('Got response:', response);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

Nice work, Ron — that's the raw Response object showing up ☑

Right now your chain is:

fetch(...) → Promise of a Response
.then((response) => { ... }) → you log the box
.catch(...) → handles errors
Next step: we'll return response.json() inside that first .then() so we can add a second .then() that receives the real data.

Here's the structure I want you to aim for (don't copy yet, just look):

Javascript

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    return response.json(); // this returns a Promise
  })
  .then((data) => {
    console.log('Got data:', data); // real JS object
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

Your turn:

In the editor, modify your existing code:
Inside the first .then, replace the console.log with return response.json();
Add a second .then((data) => { console.log('Got data:', data); }) after it.
Hit Run, see what prints.
Then hit Submit code so I can review it.
Javascript

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    console.log('Got data:', data);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

Beautiful chain, Ron — that's the full fetch → json → data flow working 🙌

You can now see:

First .then(response => ...) gets the Response (the box)
return response.json() starts reading & parsing the JSON body and returns another Promise
Second .then(data => ...) gets the parsed data (real JS object)

So in your code:

response is a Response object

data is a normal JS object with userId, id, title, body, etc.
Quick, focused check:

In your own words (just one short sentence):

What does .json() return — the data right away, or a Promise that resolves to the data later?

a prmoise to reurn the data later

Perfectly said ☑

.json() returns a Promise that will give you the data later, not the data immediately.

Let’s lock in the whole flow clearly, step by step:

fetch(url) → returns a Promise of Response
First .then((response) => { return response.json(); })
→ returns a Promise of parsed data
Second .then((data) => { ... })
→ here you finally have the parsed JS object

So conceptually:

Text
fetch → Response → response.json() → data object
One last tiny practice tweak:

In your current code, you log the whole data object.

Now I want you to log just the title.

In the editor:

Keep your fetch chain the same.
Inside the second .then, change:
Javascript
console.log('Got data:', data);
to:

Javascript
console.log('Post title:', data.title);
Run it, then Submit code again.

I want to see that you can treat data exactly like a normal JS object.

Javascript
```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    return response.json();
  })
  .then((data) => {
   console.log('Post title:', data.title);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```
Nice — that’s exactly how to use the parsed data 🎯

data is now just a regular JS object, so data.title works like any other property access.

You’ve now done all the key pieces in this lesson:

Saw that the first .then gets a Response, not the body.
Used .json() and know it returns a Promise, not data right away.
Added a second .then to get the parsed data.
Wrote the full fetch().then().then().catch() chain and used the JSON as a JS object.
Before we wrap this lesson:

Do you feel ready to move on to the next lesson, or do you want one more quick question about response vs data / .json() first?

Ready for the next lesson

Awesome work today, Ron 🎯

Quick recap in plain language:

fetch(url) gives you a Response (the box), not the data.
Most APIs send data as JSON (text that looks like a JS object).
response.json() returns a Promise, so you need another .then:

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    return response.json();      // Promise of data
  })
  .then((data) => {
    console.log(data.title);     // real JS object
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```

You turned the response into real usable data and accessed data.title like a pro 🚲

Whenever you’re ready, you can hit Next Lesson in the Learn tab and we’ll keep building on this.

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then((response) => {
    return response.json();
  })
  .then((data) => {
   console.log('Post title:', data.title);
  })
  .catch((error) => {
    console.error('Error:', error);
  });
```